

FraudGuard: A Multi-Agent LLM System for Real-Time Fraud Detection

DS5730 — Generative AI in Practice
Vanderbilt University, Spring 2026
Author: Alara Kaymak

a. Problem and Use Case

Credit card fraud is a serious and ongoing problem for banks and their customers. Most fraud detection systems today rely on rule-based logic — if a transaction exceeds a certain amount, or comes from an unusual location, it gets flagged. The problem is that these rules are either too strict (blocking legitimate purchases and frustrating customers) or too lenient (missing fraud patterns that don't fit a known template).

The idea behind FraudGuard was to build something closer to how a human fraud analyst actually thinks. When a fraud analyst looks at a transaction, they don't just check one thing — they consider the amount in the context of the customer's history, the location relative to where they were recently, the time of day, and how many transactions have come through recently. FraudGuard replicates this process using four specialized agents working in parallel, coordinated by an LLM supervisor.

The intended user is a financial institution's fraud operations team. The system handles the first pass — clearing obvious legitimate transactions quickly, flagging clear fraud for blocking, and surfacing the borderline cases for a human reviewer or a direct conversation with the cardholder. The output for each transaction is one of four decisions: APPROVE, REVIEW, BLOCK, or CLARIFICATION_NEEDED.

b. System Design

High-Level Architecture

When a transaction comes in through the REST API, it first passes through an XGBoost classifier that produces a fraud probability score between 0 and 1. If that score is extremely low (below 0.001), the transaction is approved automatically without involving the LLM at all — this keeps response times under 200ms for obviously legitimate purchases.

For everything else, the transaction is handed to a LangGraph supervisor agent. The supervisor dispatches four specialist agents simultaneously: one checks transaction velocity (how many recent transactions has this user made?), one checks location (does this location make geographic sense given where they were recently?), one checks spending (is this amount unusual for this user?), and one checks timing (is 3 AM at an electronics store normal for this person?). Each agent queries DynamoDB for the user's transaction history and returns a risk assessment. The supervisor then reads all four reports and decides: APPROVE, REVIEW, BLOCK, or CLARIFICATION_NEEDED.

If the decision is `CLARIFICATION_NEEDED`, a second endpoint handles a live multi-turn conversation with the cardholder to verify whether they made the purchase.

Main Components

The XGBoost classifier was trained on the Kaggle Credit Card Fraud dataset (284,807 transactions). It uses 30 features including PCA-transformed transaction features, scaled amount, and scaled time. IsotonicRegression calibration was applied on top so that a score of 0.73 actually reflects roughly 73% fraud probability rather than a compressed uncalibrated value.

The LangGraph supervisor uses Claude 3.5 Sonnet via Amazon Bedrock. The four specialist agents each have a DynamoDB query tool they use to pull the user’s transaction history before making their assessment. The FastAPI backend handles request routing and serves the frontend directly from the same Lambda container. The frontend is a two-column dashboard with preset demo scenarios and a custom transaction input mode.

c. Why the System is Agentic

The core of what makes this system agentic is that the LLM is making real decisions that change what happens next, not just generating text at the end of a fixed pipeline.

The most meaningful decision is the final routing decision. The supervisor receives four specialist reports that often conflict — a high velocity signal might come alongside a normal location signal, or an unusual amount might appear at a perfectly normal time of day. The supervisor has to weigh these against each other and the fraud score to decide what to do. There is no rule that tells it how to combine these signals. That judgment is entirely the LLM’s.

The `CLARIFICATION_NEEDED` path is also genuinely agentic. Once the system decides to reach out to the cardholder, Claude 3.5 Haiku manages that conversation dynamically. It reads what the cardholder says, decides whether it has enough information to make a decision, and either asks a follow-up question or resolves the case. How many turns that conversation takes depends entirely on the cardholder’s responses — the system is not following a script.

The specialist agents also use tools conditionally. When a user has no transaction history (a new user or an unknown user ID), the DynamoDB query returns nothing, and the agent has to reason about the absence of data rather than pattern-match against existing records. The velocity agent correctly reporting “no burst patterns detected” for a brand new user is not a failure — it is the right answer given what was available.

d. Technical Choices and Rationale

Claude 3.5 Sonnet was used for the supervisor and the four specialist agents because the task requires genuine reasoning about conflicting signals, not just text generation. For the cardholder conversation endpoint, Claude 3.5 Haiku was used instead — the conversation task is simpler and response speed matters more in that context.

Amazon Bedrock was the natural choice given that the rest of the infrastructure runs on AWS. It avoids managing a separate API key service and keeps IAM role-based access consistent across the system.

LangGraph was chosen because the supervisor pattern maps cleanly onto its graph abstraction. Each specialist is a node, the supervisor is a node, and state flows between them in a defined structure. This made it easy to run the four specialists in parallel and pass their results back to the supervisor in a single state update.

XGBoost is the standard approach for tabular fraud detection because it handles imbalanced data well, trains fast, and produces interpretable feature importances. The IsotonicRegression calibration step was important because uncalibrated XGBoost scores are not reliable probabilities — they tend to cluster in a narrower range than the true probability. Calibration was necessary for the score thresholds (0.001 for auto-approve, ~0.75 for block) to mean anything consistent.

DynamoDB was used for transaction history because every agent invocation queries it, so latency matters. DynamoDB's sub-10ms reads on a primary key lookup are fast enough to not meaningfully slow down the agent pipeline. The access pattern — all transactions for a given user ID — fits a simple key-value structure.

Lambda container images were required because the combined dependencies (XGBoost, scikit-learn, LangGraph, FastAPI) exceed Lambda's 250MB zip deployment limit. Container images support up to 10GB, which covers everything. Mangum wraps the FastAPI app as a Lambda-compatible handler, so the same code runs locally with uvicorn and in Lambda without any changes.

e. Observability

The observability layer uses Amazon CloudWatch Logs. Since the system runs on Lambda, all stdout and stderr output is automatically shipped to the log group `/aws/lambda/fraud-agent-api`. Structured JSON logging was added to capture two events per request.

On every incoming request, the system logs the transaction ID, user ID, amount, merchant category, city, computed fraud score, and routing decision (auto-approve vs agent). On every outgoing response, it logs the transaction ID, final decision, fraud score, routing path, and total latency in milliseconds. Classifier failures and agent errors are logged at ERROR level with the transaction ID and exception message.

This means that for any given transaction, it is possible to go into CloudWatch, find the transaction ID, and see exactly what input came in, what score was computed, whether the LLM was involved, what decision was returned, and how long it took. CloudWatch Logs Insights supports queries across all log streams, making it straightforward to pull all BLOCK decisions from the last hour or find requests that exceeded a latency threshold.

f. Metrics

Decision Accuracy

The first metric is decision accuracy: the percentage of test cases where the system’s output matches the expected decision. A 39-case labeled evaluation suite was built covering all four decision classes across a range of scenarios — obvious fraud, obvious legitimate purchases, borderline cases, impossible travel, unknown users, and edge cases like gift cards and cryptocurrency exchanges.

Accuracy came out at 82.1% (32 out of 39 cases correct). The seven failures broke down as follows: five were velocity and duplicate pattern cases where the test users had no burst history in DynamoDB, which meant the velocity agent had nothing to flag; one was a high-fraud case at score 0.73 where the agent chose REVIEW instead of BLOCK; and one was an impossible travel case at score 0.70 where the agent preferred CLARIFICATION_NEEDED over an outright BLOCK. On the clearest fraud cases — all five cases with scores above 0.80 — the system returned BLOCK every single time.

Decision Consistency

The second metric is decision consistency: given the same input, does the system return the same decision across multiple runs? This matters because LLMs are non-deterministic, and a system that gives different answers to the same question is unreliable.

Six representative transactions were each run three times and the results compared. The results showed a clear pattern: high-confidence cases were perfectly stable (100% consistency for scores above 0.80 and below 0.10), while borderline cases showed expected variance (67% consistency for scores in the 0.35–0.45 range). One important finding was that fraud scores themselves were perfectly deterministic across all runs — the XGBoost model always returned the same number for the same input. The non-determinism was entirely in the LLM’s final decision label, and only at the boundary between decision categories.

Additional metrics tracked during evaluation included p95 latency (13,988ms), average latency (9,576ms), auto-approve path latency (196ms average), and hallucination rate on unknown users (0 out of 3 runs invented transaction history for a user with no records).

g. Evaluation

Test Setup

The evaluation ran 39 labeled transactions against the live deployed Lambda endpoint. Cases covered: clear fraud (high scores, unusual times and locations), clear legitimate purchases (known high-spending user in their home city), card testing velocity patterns, impossible geographic travel, borderline cases where the right answer is genuinely ambiguous, new users with no transaction history, and high-risk merchant categories like cryptocurrency exchanges, wire transfers, and gift cards.

Where It Worked Well

The system handled the two ends of the spectrum reliably. Every high-confidence fraud case (score above 0.80) was blocked, and every case involving the known high-spending user (user_002, who regularly makes large electronics purchases in San Francisco) was approved. The system correctly recognized that a \$3,500 Apple Store purchase is normal for that user while a \$1,250 electronics purchase at 2 AM in a city they have never been to is not.

Explanation quality was also strong. In five out of six consistency test cases, the LLM explanation specifically cited the signals that drove the decision — mentioning the fraud score, the time of day, the location, or the spending pattern. This matters because an opaque decision is harder to audit than one that shows its reasoning.

The unknown user behavior was notably good. When the system was given user_999, who has no transaction history at all, it never invented a history. It acknowledged the lack of data and defaulted to CLARIFICATION_NEEDED or REVIEW, which is the appropriate conservative response.

Where It Struggled

The five velocity and duplicate pattern failures were all caused by missing data. The test users had no burst transaction history in DynamoDB, so the velocity agent correctly reported that no unusual patterns were detected — and was wrong as a result. This is a data problem rather than a model problem, but it exposed an important real-world concern: the system cannot distinguish between a new legitimate user and someone testing a stolen card if neither has prior history.

The borderline inconsistency was the other notable finding. At scores between 0.35 and 0.55, the supervisor oscillates between REVIEW and CLARIFICATION_NEEDED across runs. This is a consequence of LLM temperature — at the boundary between categories, small random variations in the generation can tip the decision either way. This is inherent to the approach and not easily fixed without either removing LLM judgment from that range (replacing it with a rule) or accepting the variance.

Tradeoffs

The system is calibrated to prioritize catching clear fraud over avoiding false positives. At scores above 0.80, it always blocks. At the boundary, it prefers CLARIFICATION_NEEDED over a definitive block, which means borderline cases go to the cardholder rather than being blocked outright. For a bank, the cost of missing obvious fraud is higher than the cost of occasionally asking a cardholder to confirm a purchase, so this is a reasonable tradeoff.

The 10-second average latency is the biggest practical constraint. It is fine for online transactions where the user submits a form and waits, but it would not work for a point-of-sale terminal where customers expect a 1–2 second response.

h. Deployment

The system is deployed on AWS Lambda as a container image, exposed publicly through Amazon API Gateway.

Public URL: <https://wysewao87f.execute-api.us-east-2.amazonaws.com>

The container image is built from the Lambda Python 3.12 base image and stored in Amazon ECR. The Lambda function runs with 1024MB of memory and a 120-second timeout. Transaction history and decision logs are stored in two DynamoDB tables. The LLM calls go to Amazon Bedrock in us-east-2.

Several practical constraints shaped the deployment. Lambda Function URLs were blocked by an organizational IAM policy, so API Gateway was used as the public endpoint instead. Docker buildx by default produces a multi-platform manifest list, which Lambda rejects — the build command required the `--provenance=false` flag to produce a single-architecture image. XGBoost also requires `libgomp` for OpenMP threading, which is not included in the Lambda base image and had to be installed explicitly in the Dockerfile.

The only environment variables the function needs are the DynamoDB table names. AWS credentials are provided automatically by the Lambda execution role.

i. Reflection

What I Learned

The most useful thing I learned from this project is that LLM non-determinism is not random in the way I expected. I thought inconsistency would show up everywhere. Instead, it showed up almost entirely at decision boundaries — cases where the input was genuinely ambiguous and a human analyst might also be unsure. High-confidence cases (score 0.92, 3 AM, jewelry purchase in a city the user has never visited) were rock solid across runs. The inconsistency was concentrated exactly where you would expect uncertainty to appear.

The velocity failures also taught me something I will not forget: an agent is only as good as the data its tools can access. The velocity agent is well-designed. The problem was that it had nothing to look at. This applies to any agentic system — you can build excellent reasoning on top of poor data and still get wrong answers.

One infrastructure lesson that turned out to matter more than I expected: serving the frontend directly from the Lambda container (same origin as the API) eliminated all the CORS complexity that would come from having a separately hosted frontend. It also simplified deployment to a single artifact. For a project of this size, that simplification was worth more than the flexibility of having them separate.

What I Would Improve

If I had more time, the first thing I would do is seed DynamoDB with realistic synthetic transaction histories for the test users. The velocity and pattern detection failures would almost certainly re-

solve once the agents have real data to work with, and I would expect accuracy to climb from 82% to above 90%.

I would also add streaming for the specialist agent results. Right now the frontend shows nothing for 10 seconds while all four agents complete. If each specialist's result appeared on screen as it finished, the system would feel much more responsive and users could see the analysis happening in real time.

On the observability side, CloudWatch captures what I needed for this project, but LangSmith would give significantly more insight in production — individual agent call traces, token counts per step, and latency breakdowns within the graph. That level of detail would make debugging a strange supervisor decision much faster.

Design Choices I Would Revisit

Running all four specialist agents in parallel for every routed transaction means that a score of 0.38 triggers the same full agent pipeline as a score of 0.95. A staged approach — run the cheapest check first and only escalate to additional agents if the signal is ambiguous — would cut latency and cost for the large portion of transactions that are probably legitimate.

I would also reconsider how aggressively the system uses `CLARIFICATION_NEEDED`. In the evaluation it worked well as a safety valve, but in a real deployment, sending a cardholder a verification request for every borderline case would generate a lot of friction. There needs to be a budget — a limit on how often a user can be contacted — and a way to decide whether a particular borderline case is worth the interruption.